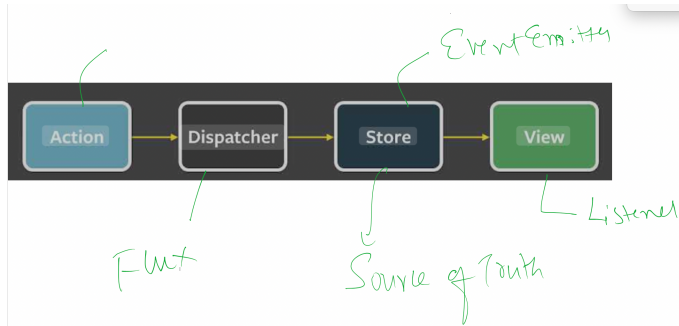
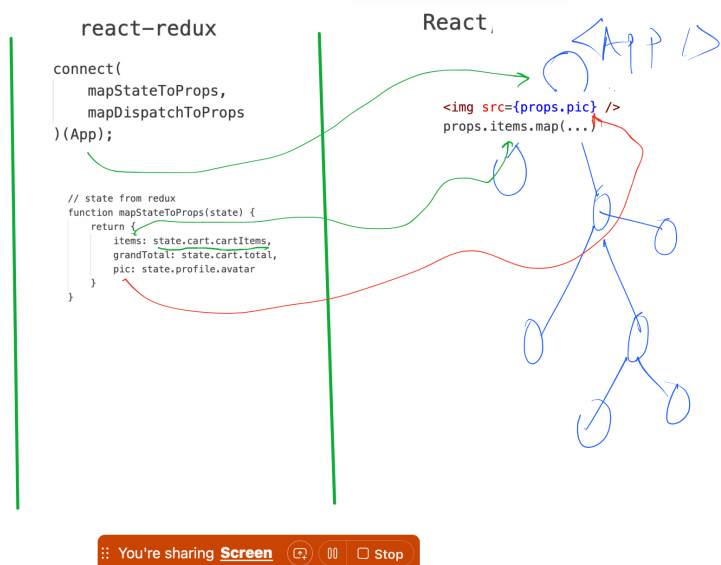
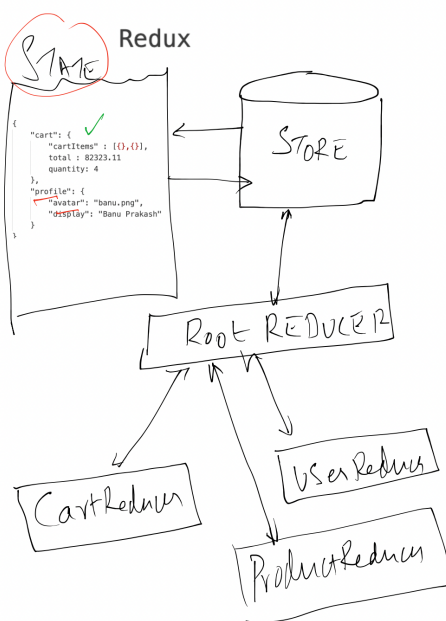
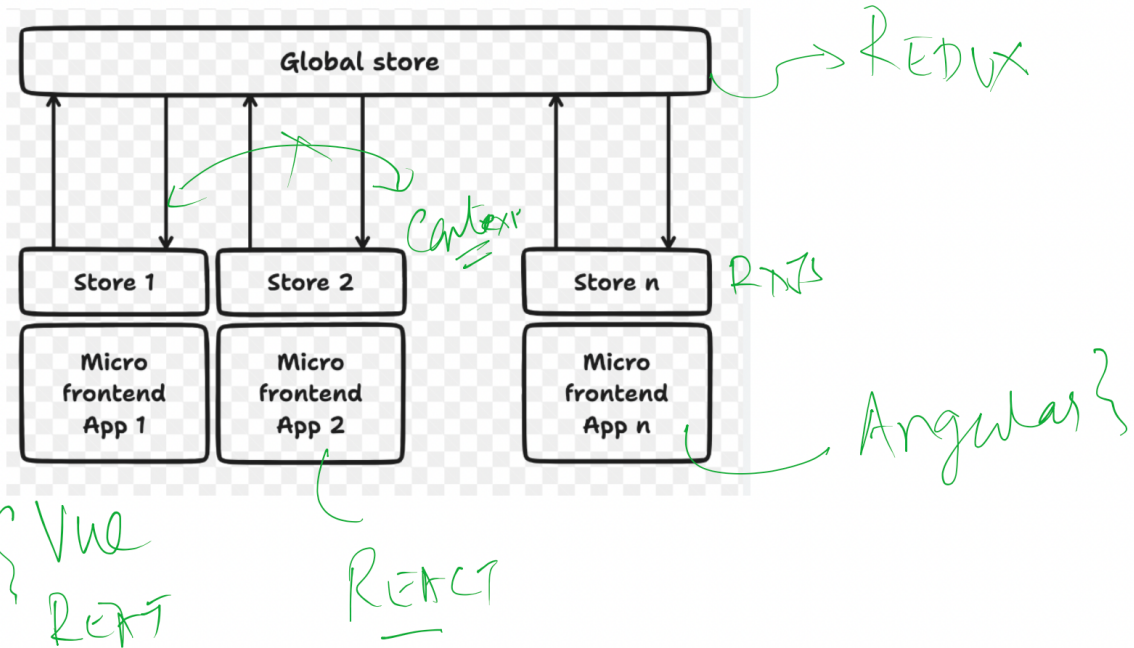
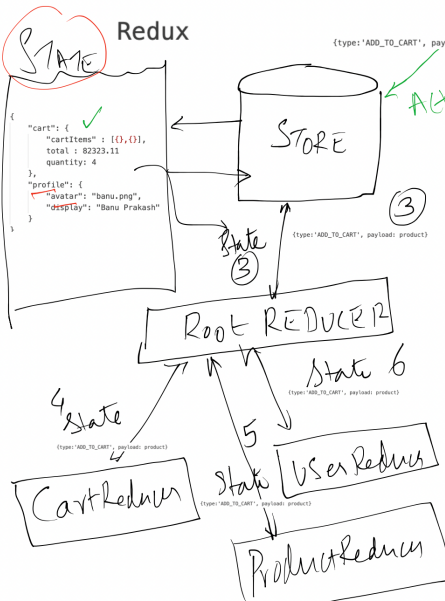




MICRO FRONTEND





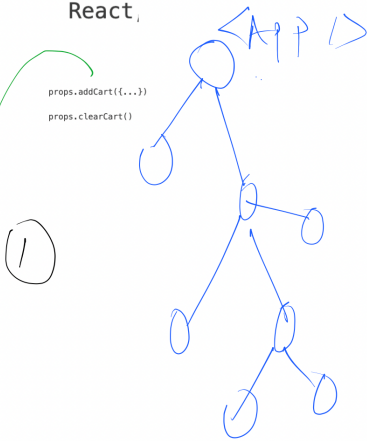
react-redux

```

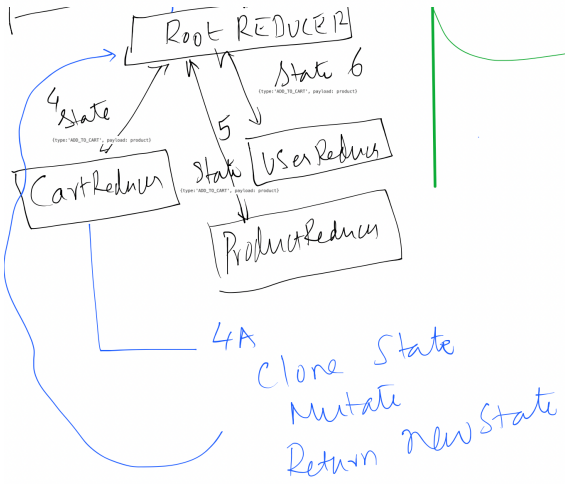
connect(
  mapStateToProps,
  mapDispatchToProps
)(App);

// dispatch action to store
function mapDispatchToProps(dispatch) {
  return {
    addCart: product => dispatch({type: 'ADD_TO_CART', payload: product}),
    clearCart: () => dispatch({type: 'CLEAR_CART'})
  }
}

```



You're sharing **Screen** [Icons] Stop



```

const store = createStore(rootReducer);
export default store;

export const rootReducer = combineReducers({
  profile: profileReducer,
  contacts: contactReducer
});

export default function profileReducer(state = {
  "avatar": "banu.png", "name": "Banu Prakash"
}, action) {
  return state;
}

export default function contactReducer(state = [], action) {
  switch(action.type) {
    case 'ADD_CONTACT':
      // state.push(action.payload); // not change original state
      // clone state and add payload to the clone and return
      return [...state, action.payload];
    case 'REMOVE_CONTACT':
      return state.filter(contact => contact.email !== action.payload);
    case 'CLEAR_CONTACTS':
      return [];
    default:
      return state;
  }
}

```

```
const store = createStore(rootReducer);  
export default store;
```

```
STATE IN STORE:  
{  
  contacts: [],  
  profile: {  
    "avatar": "banu.png", "name": "Banu Prakash"  
  }  
}
```

```
export default function profileReducer(state = {  
  "avatar": "banu.png", "name": "Banu Prakash"  
}, action) {  
  return state;  
}
```

```
createRoot(document.getElementById('root')).render(  
  <Provider store={store}>  
    <App />  
  </Provider>  
)
```

```
export const rootReducer = combineReducers({  
  profile: profileReducer,  
  contacts: contactReducer  
});
```

```
export default function contactReducer(state = [], action) {  
  switch(action.type) {  
    case 'ADD_CONTACT':  
      // state.push(action.payload); // not change original state  
      // clone state and add payload to the clone and return  
      return [...state, action.payload];  
    case 'REMOVE_CONTACT':  
      return state.filter(contact => contact.email !== action.payload);  
    case 'CLEAR_CONTACTS':  
      return [];  
    default:  
      return state;  
  }  
}
```

```
export default connect(  
  mapStateToProps = (state) => {  
    {  
      pic: state.profile.avatar, ✓  
      displayName: state.profile.name, ✓  
      contactList: state.contacts  
    }  
  },  
  mapDispatchToProps = (dispatch) => ({  
    addContact: (contact) => dispatch({ type: 'ADD_CONTACT', payload: contact }),  
    removeContact: (email) => dispatch({ type: 'REMOVE_CONTACT', payload: email }),  
    clearContacts: () => dispatch({ type: 'CLEAR_CONTACTS' })  
  })  
) (App)
```

pic
displayName